OOPS

Saturday, 3 January 2026 9:55 AM

coding languages
- → object oriented → we model the problem as classes & interactions b/w them
- → procedural / functional
  - ↳ we model as functions which do some computation & return the result.

mutation

method → pass object
↓
inside method
↳ we change the object's properties

① method1 → method2 → change in object

② method1 → method 2
↓ values
set the values.

OOPS

Object Oriented Programming Paradigm

model the system as classes & their interactions.

For example: Designing Library

- → Library → add book / remove a book / issue a book
- → Book
- → User → get book / return book
- → Librarian → aid in library operations

① Classes
↳ user defined data type

- some variables
- some methods
- template
- does not hold any memory in heap

```
class Book {
    private String name;
    private String author;
    private int regNumber;
}
```

② Objects

- object is the basic unit of OOP
- it is the actual memory allocation based on the class.
- variables are the state of object
- methods are the behavior.

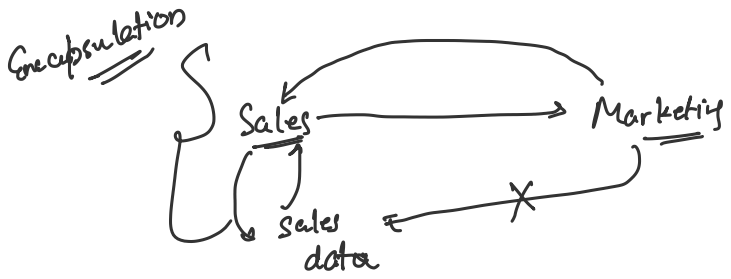
③ Encapsulation

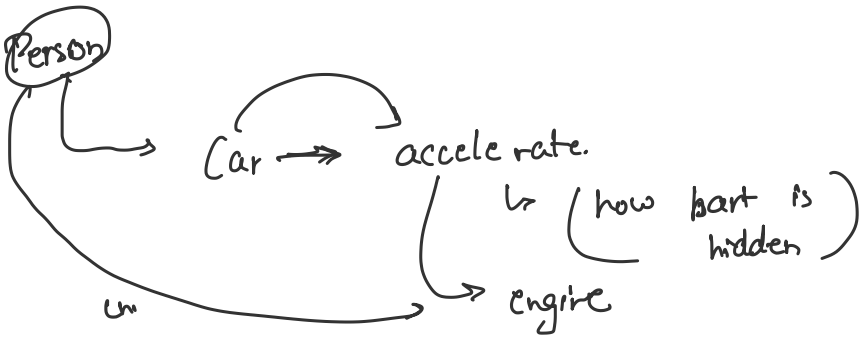
- wrapping data & methods in a single unit.
- binds the data & code together.
- variables of a class are hidden from other classes & can be accessed through a member function only.
- This hiding of data members is also called data hiding.

④ Abstraction

- hiding the details
- hide the "what" part.
- provide essential information without providing details

	En capsulation	Abstraction.
focus	"how"	"what"
goal	protect data	simplify complexity
how to do	using access modifiers.	using interfaces & abstract classes.
what to hide	hides internal state	hides implementation details



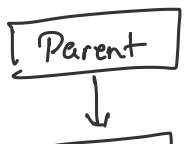


⑤ Inheritance (IS-A) relation

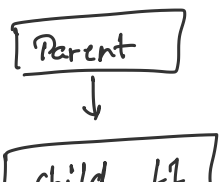
- capability to derive properties & characteristics from another class
- aimed to reuse the codes & reduce redundancy.

Types of Inheritance.

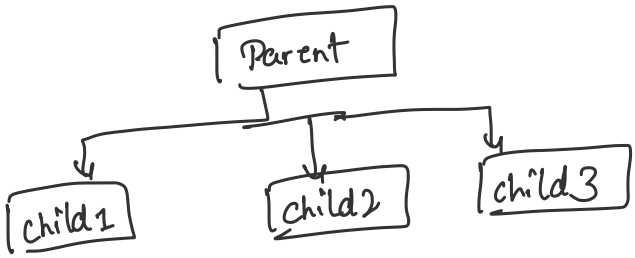
① Single inheritance



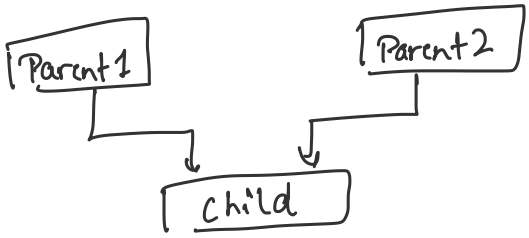
② Multi level inheritance



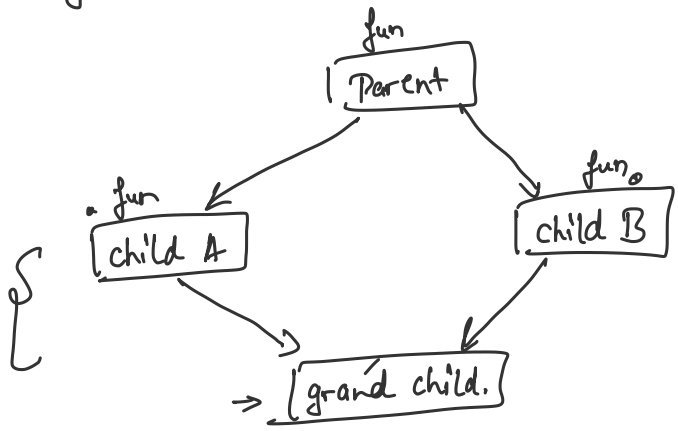
③ Hierarchical Inheritance.



④ Multiple inheritance.



⑤ Hybrid or Diamond.



C++ supports all kinds of inheritance.
Multiple & Hybrid inheritance can cause "ambiguity" issues when both class have same method name.
It is resolved by using scope resolution (class-name ::)

Java does not support multiple & hybrid inheri tance.

⑥ Polymorphism

↳ many forms

Compile-time

↳ function overloding

→ 2 or more functions with the same name.

→ differentiated on the number & type of arguments.

number

add (a, b)

add (a, b, c)

type

add (int a, int b)

add (double a, double b)

obj. add

operator overloading

int c = a+b;

what if the arguments are objects.

Book c = a + b

Java

int a = b+c → add

String a = b+c → concat

char a = (b+c) → add & convert to char

Runtime
↳ inheritance

Book
- Fiction Book
- Fantesy Book

Parent → Book book = new Fiction Book(); (child)

book.method();

Libraery
↳ issue (Book book)

Book
- Fiction Book
- Fantesy Book

issue (Fiction
issue (Fantesy
⋮

Nested class

Person
↳ Address
↳ variables

Association (HAS-A) relation

Association

- generic term
- whenever one class contains another, it is association
- 1:1, 1:M, M:M
- it does not indicate ownership

Aggregation

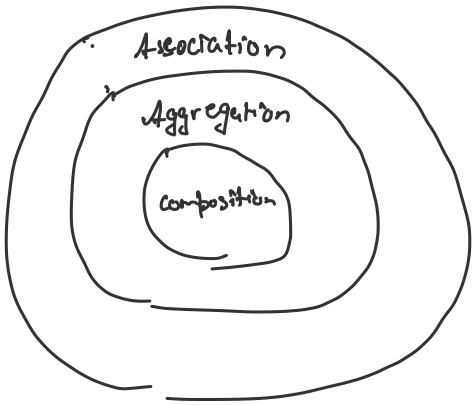
- one class contains a collection of other class

  Example: Library & book
- Both the classes can have independent existence.

Composition

- stronger form of aggregation
- indicates ownership
- Inner class cannot exist independently.

Example. house & rooms



Library

Library → Book (Aggregation)

Person → Book (Association)